Backend route excerpt (continued) *app.py*

```
@app.route("/api/mode", methods=["POST"])
def update_mode():
    payload = request.get_json(silent=True) or {}
    value = payload.get("simulation")
    if not isinstance(value, bool):
        return jsonify({"error": "simulation must be true or false"}), 400
    try:
        return jsonify(set_simulation_mode(value))
    except Exception as exc:
        app.logger.exception("Could not switch data source mode")
        return jsonify({"error": str(exc) or "Could not switch mode"}), 500


@app.route("/api/logging/status")
def logging_status():
    source_status = get_status()
    return jsonify(
        {
            "icaoExcelEnabled": bool(getattr(flight_store, "icao_excel_enabled", False)),
            "icaoExcelDir": str(getattr(flight_store, "icao_excel_dir", "")),
            "icaoMasterEnabled": bool(getattr(flight_store, "icao_master_enabled", False)),
            "icaoMasterPath": str(getattr(flight_store, "icao_master_path", "")),
            "matrixCsvEnabled": matrix_csv_enabled,
            "matrixCsvPath": matrix_csv_path,
            "sessionDir": str(SESSION_DIR),
            "hexFile": source_status.get("hexFile"),
            "asciiFile": source_status.get("asciiFile"),
            "rawFile": source_status.get("rawFile"),
        }
    )


@app.route("/api/training/trajectory")
def training_trajectory():
    flight_key = request.args.get("flightKey")
    hours = int(request.args.get("hours", "24"))
    limit = int(request.args.get("limit", "200000"))
    rows = flight_store.get_trajectory_history(flight_key=flight_key, hours=hours, limit=limit)
    return jsonify({"rows": rows, "count": len(rows), "flightKey": flight_key or "ALL", "hours": hours})


@app.route("/api/training/trajectory/export")
def training_trajectory_export():
    flight_key = request.args.get("flightKey")
    hours = int(request.args.get("hours", "24"))
    limit = int(request.args.get("limit", "200000"))
    stamp = datetime.utcnow().strftime("%Y%m%dT%H%M%SZ")
    flight_tag = (flight_key or "ALL").upper()
    out_path = (
        app_dir()
        / "logs"
        / "exports"
        / f"trajectory_{flight_tag}_{hours}h_{stamp}.csv"
    )
    csv_path, row_count = flight_store.write_trajectory_history_csv(
        output_path=out_path,
        flight_key=flight_key,
        hours=hours,
        limit=limit,
    )
    return jsonify({"csvPath": csv_path, "rowCount": row_count, "flightKey": flight_tag, "hours": hours})

if __name__ == "__main__":
    port = int(os.getenv("ADSB_PORT", "5005"))
    if os.getenv("WERKZEUG_RUN_MAIN") != "true":
        threading.Timer(1.5, lambda: webbrowser.open(f"http://localhost:{port}/")).start()
    app.run(host="localhost", port=port, debug=False)
```